Buổi 1 & 2 – CineJava v1: Quản lý vé xem phim

Nov 4, 2026 · @Trần Văn Huy

Công cụ trước khi bắt đầu:

JDK 21,25,27 (Cài đặt môi trường cho java trên windows)
Visual Studio code
Các extension cho Vscode: java(oracle), extension pack for java(microsoft)

Tạo thư mục MOB1024 trên ổ D\E(nếu không có dùng C)

Mục tiêu

Viết chương trình Java quản lý vé xem phim của rạp CineJava, áp dụng **đóng gói, kế thừa, đa hình và xử lý ngoại lệ**.

Sau bài lab, sinh viên làm được:

- Khai báo lớp có thuộc tính `private`, getter và setter có kiểm tra dữ liệu.
- Tạo lớp con bằng `extends`, gọi constructor lớp cha bằng `super(...)`.
- Ghi đè phương thức bằng `@Override` để mỗi lớp con tính giá theo cách riêng.
- Ném và bắt ngoại lệ bằng `throw` và `try / catch`.
- Chia code thành package và các hàm nhỏ, đặt tên đúng quy ước Java.

Mô tả bài toán

Rạp CineJava bán ba loại vé. Vé nào cũng có **mã vé**, **tên phim** và **giá gốc**. Mỗi loại vé có một công thức riêng để tính **giá khách phải trả**:

Loại vé	Thông tin thêm	Giá khách phải trả	Ví dụ
Vé thường	Không có	Giá gốc	90.000 → 90.000
Vé VIP	Tỉ lệ phụ thu (0 → 1), phí combo bắp nước (≥ 0)	Giá gốc + giá gốc × tỉ lệ phụ thu + phí combo	110.000, phụ thu 0,2, combo 30.000 → 162.000
Vé sinh viên	Phần trăm giảm giá (0 → 50)	Giá gốc × (100 − % giảm) / 100	90.000, giảm 20% → 72.000

Chương trình cần: tạo danh sách vé, in danh sách, tìm vé đắt nhất, từ chối dữ liệu sai, và đọc thêm vé từ các dòng văn bản.

Cấu trúc project

Tạo project Java mới trong VS Code (**Java: Create Java Project → No build tools**), đặt tên `slide-1`. Sau đó tự tạo các package và file theo đúng cấu trúc sau:

```
slide-1/
└── src/
    └── cinejava/
        ├── Main.java
        ├── model/
        │   ├── Ticket.java
        │   ├── VipTicket.java
        │   └── StudentTicket.java
        └── service/
            └── TicketService.java
```

- Dòng `package` ở đầu mỗi file phải khớp thư mục: `package cinejava.model;` cho các file trong `model/`.
- Class dùng PascalCase, phương thức và biến dùng camelCase, tên viết bằng tiếng Anh.
- Giữ đúng tên class, phương thức và thuộc tính như đề bài yêu cầu, vì giảng viên chấm theo các tên này.

Bài 1 – Lớp Ticket

Tạo lớp `Ticket` trong package `cinejava.model`, đại diện cho vé thường.

Thuộc tính (đều là `private`):

Tên	Kiểu	Ý nghĩa
`id`	`String`	Mã vé
`movie`	`String`	Tên phim
`basePrice`	`double`	Giá gốc

Yêu cầu:

1. Viết getter cho cả 3 thuộc tính: `getId()`, `getMovie()`, `getBasePrice()`.
2. Viết setter có kiểm tra. Dữ liệu sai thì ném `IllegalArgumentException` với đúng thông báo sau:
    - `setId(String id)`: rỗng hoặc `null` → `"Mã vé rỗng"`. Hợp lệ thì lưu sau khi `trim()`.

- `setMovie(String movie)`: rỗng hoặc `null` → `"Tên phim rỗng"`. Hợp lệ thì lưu sau khi `trim()`.
- `setBasePrice(double p)`: nhỏ hơn 0 → `"Giá vé phải >= 0"`.

3. Viết constructor `Ticket(String id, String movie, double basePrice)`. Constructor **phải gọi các setter** ở trên, không được gán thẳng `this.id = id;`. Nhờ vậy mọi đối tượng tạo ra đều hợp lệ.
4. Viết phương thức `double finalPrice()` trả về giá khách phải trả. Với vé thường, giá này bằng giá gốc.
5. Viết phương thức `void printInfo()` in vé trên một dòng theo định dạng dưới đây. Phải gọi `finalPrice()`, không dùng thẳng `basePrice`, để lớp con in đúng giá mà không cần viết lại.

```
System.out.printf("%-5s | %-12s | %,10.0f%n", id, movie, finalPrice());
```

Câu hỏi: vì sao `printInfo()` gọi `finalPrice()` mà không dùng `basePrice`? Nếu dùng `basePrice` thì vé VIP sẽ in ra giá bao nhiêu?

Bài 2 – Lớp VipTicket và StudentTicket

Tạo hai lớp con kế thừa `Ticket`, cũng đặt trong package `cinejava.model`.

VipTicket

Thuộc tính	Kiểu	Ràng buộc	Thông báo lỗi
`surchargeRate`	`double`	Từ 0 đến 1	`"Phụ thu phải trong khoảng 0–1"`
`comboFee`	`double`	≥ 0	`"Phí combo phải >= 0"`

1. Viết getter, setter có kiểm tra cho hai thuộc tính trên.
2. Constructor `VipTicket(String id, String movie, double basePrice, double surchargeRate, double comboFee)`: dòng đầu tiên gọi `super(id, movie, basePrice)`, sau đó gọi `setSurchargeRate(...)` và `setComboFee(...)`.
3. Ghi đè `finalPrice()`: giá gốc + giá gốc × `surchargeRate` + `comboFee`. Vì `basePrice` là `private` của `Ticket`, hãy đọc qua `getBasePrice()`.

StudentTicket

Thuộc tính	Kiểu	Ràng buộc	Thông báo lỗi
`discountPercent`	`int`	Từ 0 đến 50	`"Giảm giá phải trong khoảng 0–50%"`

1. Viết getter, setter có kiểm tra cho `discountPercent`.
2. Constructor `StudentTicket(String id, String movie, double basePrice, int discountPercent)`: gọi `super(...)` rồi gọi `setDiscountPercent(...)`.
3. Ghi đè `finalPrice()`: giá gốc × (100 − `discountPercent`) / 100.

Cả hai lớp con **không được** viết lại `printInfo()`. Chạy thử và kiểm tra vé VIP vẫn in đúng giá sau phụ thu.

Bài 3 – TicketService và Main (2 điểm)

Tạo lớp `TicketService` trong package `cinejava.service`, chứa các hàm `public static` sau:

Hàm	Yêu cầu
`List<Ticket> createSampleTickets()`	Trả về danh sách 5 vé mẫu ở bảng dưới
`void printAll(List<Ticket> tickets)`	In từng vé bằng vòng `for`, gọi `printInfo()`
`Ticket findMostExpensive(List<Ticket> tickets)`	Trả về vé có `finalPrice()` lớn nhất
`void addInvalidTickets(List<Ticket> tickets)`	Thử thêm 2 vé sai dữ liệu ở bảng dưới. Mỗi vé bọc trong một `try / catch` riêng, bắt `IllegalArgumentException` và in `[LỖI]` + thông báo lỗi

Dữ liệu mẫu:

Loại	Mã	Phim	Giá gốc	Tham số thêm
Thường	T01	Mai	90.000	
VIP	V01	Dune	110.000	phụ thu 0,2; combo 30.000
Sinh viên	S01	Mai	90.000	giảm 20%
Thường	T02	Exhuma	100.000	
VIP	V02	Inside Out 2	90.000	phụ thu 0,1; combo 25.000

Vé sai dữ liệu dùng cho `addInvalidTickets`:

Loại	Mã	Phim	Giá gốc	Tham số thêm	Lỗi mong đợi
Sinh viên	S09	Dune	90.000	giảm 70%	Giảm giá vượt 50%
Thường	T09	Mai	−5		Giá âm

Trong `Main.java` (package `cinejava`), hàm `main` chỉ gọi các hàm của `TicketService` theo thứ tự:

1. Tạo danh sách vé mẫu.
2. In tiêu đề `===== DANH SÁCH VÉ =====` rồi in tất cả vé.
3. In `Vé đắt nhất:` rồi in vé đắt nhất trên cùng dòng.
4. In tiêu đề `===== DỮ LIỆU LỖI =====` rồi gọi `addInvalidTickets`.
5. Phần nhập vé từ chuỗi (Bài 4).
6. In `Tổng số vé:` + số vé trong danh sách.

Bài 4 – Nhập vé từ chuỗi

Thêm vào `TicketService` hai hàm để đọc vé từ các dòng văn bản có dạng `loại;mã;phim;giá;tham số`.

Hàm `Ticket parse(String line)` tách chuỗi bằng `split(";")` và tạo vé theo loại. Dùng `if / else if / else` để chọn loại:

Loại	Tạo vé	Tham số thứ 5
`VIP`	`VipTicket`, phí combo cố định 30.000	Tỉ lệ phụ thu (`double`)
`STUDENT`	`StudentTicket`	Phần trăm giảm (`int`)
Loại khác	`Ticket`	Bỏ qua

Hàm `void importFromLines(List<Ticket> tickets, String[] lines)` duyệt từng dòng và gọi `parse`:

- Đọc được thì thêm vào danh sách, in `OK` rồi in vé trên cùng dòng.
- Gặp `NumberFormatException` (giá hoặc tham số không phải số) thì in `[LỖI SỐ]` + dòng đó.
- Gặp `IllegalArgumentException` (dữ liệu sai) thì in `[LỖI]` + dòng đó + `→` + thông báo lỗi.
- Một dòng lỗi không được làm dừng chương trình.

Trong `main`, in tiêu đề `===== NHẬP VÉ TỪ CHUỖI =====` rồi gọi `importFromLines` với dữ liệu sau:

```
String[] lines = {
        "VIP;V03;Mai;100000;0.15",
        "STUDENT;S02;Dune;abc;10",   // giá là chữ
        "NORMAL;;Mai;90000;0",       // mã vé rỗng
        "STUDENT;S03;Exhuma;95000;30"
};
```

Câu hỏi: `NumberFormatException` là lớp con của `IllegalArgumentException`. Nếu đặt `catch (IllegalArgumentException e)` lên trước `catch (NumberFormatException e)` thì chuyện gì xảy ra? Giải thích.

Kết quả chạy mong đợi

Chương trình đúng phải in ra **2 lỗi** ở phần DỮ LIỆU LỖI và kết thúc với **Tổng số vé: 7** (5 vé mẫu + 2 vé đọc được từ chuỗi):

```
===== DANH SÁCH VÉ =====
T01   | Mai           |     90,000
V01   | Dune          |    162,000
S01   | Mai           |     72,000
T02   | Exhuma        |    100,000
V02   | Inside Out 2  |    124,000
Vé đắt nhất: V01   | Dune          |    162,000

===== DỮ LIỆU LỖI =====
[LỖI] Giảm giá phải trong khoảng 0–50%
[LỖI] Giá vé phải >= 0

===== NHẬP VÉ TỪ CHUỖI =====
OK   V03   | Mai           |    145,000
[LỖI SỐ] STUDENT;S02;Dune;abc;10
[LỖI] NORMAL;;Mai;90000;0 → Mã vé rỗng
```

```
OK   S03   | Exhuma       |     66,500
Tổng số vé: 7
```

- Phần DỮ LIỆU LỖI trống, hoặc vé mã rỗng báo `OK`, nghĩa là constructor chưa gọi setter. Xem lại Bài 1, yêu cầu 3.
- In ra dạng `cinejava.model.Ticket@7344699f` nghĩa là bạn đang `println(ticket)` thay vì gọi `ticket.printInfo()`.
- Dấu phân cách hàng nghìn có thể là `.` thay vì `,` tuỳ cài đặt ngôn ngữ của máy. Cả hai đều được chấp nhận.
- Tiếng Việt bị lỗi font trong terminal thì chạy với `java "-Dstdout.encoding=UTF-8" ...`.